

may appear to be straightforward, computers have difficulty generating random numbers. This is because computers are programmed to obey instructions, which is the polar opposite of randomness. Giving instructions on choosing something randomly is an oxymoron therefore you can't just order a computer to "come up with a random number." As a result, most video games use something called Pseudo-RNG. Pseudo-RNG generates a random number using an algorithm that conducts



All Hail the RNGesus

mathematical operations on a beginning value called a seed. It's critical to select a seed as random as possible to get different results each time. Using the same seed will produce the same results.

Game designers primarily apply RNG in games in two ways:

Input Randomness

This sort of randomisation provides information to the user before making a decision. Think of procedurally generated dungeons and maps in roguelike games or telegraphing information about the enemy in games like Slay the Spire, or Into the Breach. Input randomness gives players a chance to react to an unpredictable event.

Output Randomness

After a decision is made, random information is introduced into the

game. In games like X-Com, you pick whether or not to shoot the alien, and then a dice roll/random number generator determines whether or not you hit. To put it another way, with output randomness, the player has zero time to "do anything" with the new information that has been provided.

Utilising randomness in a game can either make or break the gameplay. Randomness is a very abstract idea that can be hard for gamers to digest. Once we've developed a relationship with these abstract data points, they begin to matter and signify something. In games, information takes on meaning based on its relationship to other components of the game state, such as the history of game states. But most importantly, it takes on purpose based on your input. The chances, such as having an 81 percent probability of hitting, have a more profound systemic significance since they are linked to how you trained your character, where you positioned him, what weapon you provided him, and so on. But the fact that he misses or hits himself – that boolean flag of 0 or 1 – has no systemic significance; it has no connection to the gamers at all. Players don't want items that appear to permanently alter the game state because they haven't had time to build a relationship with it. This is why input unpredictability is beneficial, but output randomness is detrimental. However, if used creatively, even output randomness can make the gameplay more fun. Using a system that does not depend on boolean states, but rather several different states, each with a different set of

effects and outcomes would make gameplay seem more fair to player. Also, no one would complain if output randomness worked in their favour (Eg. If enemies miss attacks due to output randomness).

Indie Game Development:

As games started settling down into specific templates, there was a certain expectation from most video games: the duration played should average anywhere between 30-40 hours. This has slowly increased to the industry standard of 60 hours in today's age, which indie developers can't hope to compete with. Let's dial back and talk about indie development first.

Deriving from "independent", the indie game development scene typically has the same hallmarks of a startup. Limited workforce, budget, and time, but unlimited creativity. Where well-established studios have to stick to the norm, indie developers can break free of the mould to try new and exciting ideas and mechanics. But they have to be clever about it, since resources are scarce. Games that have been considered the darlings and underdogs of the indie video game scene, like Fez and Braid, took five and three years, respectively. This was very expensive and stressful for the developers. Indie studios realised that this model could be scalable only if they have the backing of a small-medium sized team or they need to rely on various tricks and techniques to extend gameplay time without necessarily hand crafting all the content for it. This most often assumes using random-

ness, or RNG for deciding certain parameters, such as configurations of enemies you face, or the loot that you may find. But when this principle is dialled up to eleven, we get to see it in a form which is practically unthinkable for the Triple A industry.

What if you could generate entire worlds using the power of generating



Output randomness at work